

Scalable Low-Input-Dimensional Multi-Output Gaussian Process Inference via State-Space Message Passing

Anonymous Authors

identities and affiliations suppressed for double-blind review

Abstract

Multi-output Gaussian process (MOGP) inference with D correlated outputs and N observations can require $\mathcal{O}((DN)^3)$ kernel-matrix operations, which is prohibitive in sequential design loops. We formulate low-input-dimensional MOGP regression as state-space message passing on a Forney-style factor graph. A nearest-neighbor chain maps a fixed candidate set of size C onto a one-dimensional sequence, while the Linear Model of Coregionalization provides multi-output structure with per-output scalar observation noise. For fixed hyperparameters and chain, the fully linear-Gaussian model supports single-pass inference at $\mathcal{O}(CDL)$ cost per design step, where L is the number of latent GPs, and handles partial observations by omitting local observation messages rather than restructuring covariance matrices. On a one-dimensional Bayesian quadrature task, the state-space approximation obtains lower integral error and RMSE than the kernel-matrix baseline in our implementation; on a four-dimensional Bayesian optimization task, it trades weaker posterior fidelity for an order-of-magnitude reduction in computation, with final best values close to random search and below the kernel-matrix baseline.

Preliminary work. Under review by ProbNum 2026. Copyright with the authors. Do not distribute.

1 Introduction

Many sequential-design problems require posterior inference for expensive functions with multiple correlated outputs. Bayesian optimization (BO) is one example: it uses a probabilistic surrogate and an acquisition function to balance exploration and exploitation (Snoek et al., 2012; Frazier, 2018; Garnett, 2023). Gaussian processes (GPs) are common surrogates because they provide flexible uncertainty quantification (Rasmussen and Williams, 2006). Many real-world sequential-design and inference tasks involve multiple correlated output quantities. In environmental monitoring, for example, pollutant concentrations measured at different spatial locations and times share a latent physical structure. Multi-output GPs capture such cross-output correlations through techniques such as the Linear Model of Coregionalization (LMC), which expresses the observed outputs as linear mixtures of independent latent GPs (Alvarez et al., 2012). However, fitting a standard multi-output GP with D outputs and N training points can require operations on a $DN \times DN$ covariance matrix, i.e., $\mathcal{O}((DN)^3)$, which becomes prohibitively expensive when D or N grows.

State-space representations of GPs offer a well-known path to linear-time inference (Hartikainen and Särkkä, 2010; Särkkä and Solin, 2019). A GP with a Matérn covariance function can be expressed as the solution of a linear stochastic differential equation (SDE), and inference reduces to Kalman filtering and smoothing at $\mathcal{O}(N)$ cost. This connection has been widely exploited in temporal settings, where the natural ordering of time provides the one-dimensional chain required by the state-space formulation. State-space models, in turn, have an equivalent representation as Forney-style factor graphs (FFGs), on which inference is performed

by message passing (Loeliger et al., 2007). In this work, we formulate low-input-dimensional MOGP regression as factor-graph message passing. With scalar observation factors, the model is fully linear-Gaussian and inference over a fixed candidate chain is a single forward-backward pass at $\mathcal{O}(CDL)$ cost per design step (see Sec. 4.1.2). For partial observations, unobserved outputs simply omit local observation messages: the same model and inference call apply, without covariance-matrix restructuring (Baudart et al., 2020; Bagaev and De Vries, 2023). A greedy nearest-neighbor chain ordering maps a fixed low-dimensional candidate set onto a one-dimensional sequence, enabling the state-space formulation beyond temporal data while introducing a geometry-compression approximation.

We make the following contributions:

1. A factor-graph formulation of low-input-dimensional multi-output GP regression, combining state-space GPs (Hartikainen and Särkkä, 2010), FFG inference (Loeliger et al., 2007), and the LMC (Alvarez et al., 2012), with $\mathcal{O}(CDL)$ per-step inference after chain construction.
2. Native partial observations with the same model and inference call, unlike kernel-matrix methods that require covariance restructuring.
3. BQ ($M=1$, $D=6$) and environmental-monitoring BO ($M=4$, $D=12$) evaluations with sequential-design and direct posterior-quality metrics.

2 Problem statement

Let $x \in \mathcal{X} \subset \mathbb{R}^M$ be an M -dimensional input variable and let $f : \mathcal{X} \rightarrow \mathbb{R}^D$ be a D -dimensional output func-

tion. In BO applications, we select points by scalarizing the outputs with a preference vector $s \in \mathbb{R}^D$ and searching for

$$x^* = \arg \max_{x \in \mathcal{X}} s^\top f(x). \quad (1)$$

The function f is unknown or expensive to evaluate, so BO must balance exploration and exploitation.

Bayesian optimization uses a probabilistic surrogate, often a GP, to approximate f . A multi-output GP is a collection of random variables whose finite subset at N function evaluations of the D -dimensional function is matrix-normal distributed (Alvarez and Lawrence, 2011). Let $X \in \mathbb{R}^{N \times M}$ be the matrix of training inputs and $Y \in \mathbb{R}^{N \times D}$ the matrix of noisy training outputs, with $y_i = f(x_i) + \epsilon_i$ and $\epsilon_i \sim \mathcal{N}(0, \Sigma_\epsilon)$ for $\Sigma_\epsilon \in \mathbb{R}^{D \times D}$. Under a zero-mean separable MOGP prior with input covariance $K_X \in \mathbb{R}^{N \times N}$ and output covariance $K_Y \in \mathbb{R}^{D \times D}$,

$$\text{vec}(Y) \sim \mathcal{N}(0, K_Y \otimes K_X + \Sigma_\epsilon \otimes I_N). \quad (2)$$

This separable form is the special case in which all latent components share the same input-side kernel; the general LMC used in our experiments replaces $K_Y \otimes K_X$ with $\sum_{l=1}^L (W_{:,l} W_{:,l}^\top) \otimes K_l$ to allow distinct kernels k_l per latent process (see Section 3.1). The predictive distribution at a new input x_+ is Gaussian with mean vector $\mu(x_+) \in \mathbb{R}^D$ and covariance matrix $\Sigma(x_+) \in \mathbb{R}^{D \times D}$ (Rasmussen and Williams, 2006; Alvarez and Lawrence, 2011). Direct kernel-matrix inference in the fully coupled vectorized representation requires operations on an $ND \times ND$ covariance matrix, with naive cubic cost $\mathcal{O}((ND)^3)$. In BO, an acquisition function $\alpha(x)$ based on the predictive mean and variance is used to sequentially select evaluation points, $x^+ = \arg \max_x \alpha(x)$. Acquisition functions target information gain or uncertainty reduction as well as attraction to local maxima (Frazier, 2018; Garnett, 2023). Common acquisition functions such as Expected Improvement, Entropy Search and Upper Confidence Bound (Snoek et al., 2012; Hennig and Schuler, 2012; Srinivas et al., 2010), have proven to be highly useful for maximizing unknown objectives. When BO uses an exact multi-output GP surrogate, the surrogate update can become the bottleneck as observations or outputs grow. We address this by reformulating MOGP inference; Bayesian quadrature is treated as a second sequential-design application with a task-specific rule (Section 5).

3 Background

3.1 Multi-Output Gaussian Processes

Exact multi-output GP inference is costly because covariance matrices couple inputs and outputs. We focus on the Linear Model of Coregionalization (LMC), which mixes L latent GPs $g_l(x)$,

$$f_d(x) = \sum_{l=1}^L w_{dl} g_l(x) \quad (3)$$

where w_{dl} is an element of a mixing matrix $W \in \mathbb{R}^{D \times L}$. The induced cross-output covariance is

$$\text{cov}[f_d(x), f_{d'}(x')] = \sum_{l=1}^L w_{dl} w_{d'l} k_l(x, x'). \quad (4)$$

At a fixed input, or with shared latent input covariance, the output-side structure is low-rank plus diagonal noise. For $L \ll D$, Woodbury applies inverses of $WSW^\top + \Sigma_\epsilon$ through an $L \times L$ inner system, costing $\mathcal{O}(DL^2 + L^3)$, or $\mathcal{O}(DL^2)$ for fixed L , instead of dense $D \times D$ operations. The special case $L = 1$ is the Intrinsic Coregionalization Model (ICM), which is less expressive than LMC (Alvarez and Lawrence, 2011; Bonilla et al., 2007).

3.2 State-Space Gaussian Processes

The cubic complexity in samples has driven extensive work on scalable approximations. Sparse variational methods based on inducing points reduce this cost while preserving a variational lower bound on the marginal likelihood (Smola et al., 2004; Bruinsma et al., 2020). However, these are approximations. Temporal GPs with Matérn covariance functions admit an exact representation as linear SDEs, converting GP regression into Kalman filtering and smoothing at $\mathcal{O}(N)$ cost (Hartikainen and Särkkä, 2010; Särkkä, 2013; Särkkä and Solin, 2019). For example, a scalar temporal GP with a stationary Matérn 3/2 covariance function $\kappa(t, t')$, length-scale ℓ , and signal variance σ^2 can be represented exactly as the solution of the linear SDE

$$d \begin{bmatrix} g(t) \\ \dot{g}(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\lambda^2 & -2\lambda \end{bmatrix} \begin{bmatrix} g(t) \\ \dot{g}(t) \end{bmatrix} dt + \begin{bmatrix} 0 \\ 1 \end{bmatrix} dw(t), \quad (5)$$

for $\lambda = \sqrt{3}/\ell$ and white noise $w(t)$ with spectral density $4\lambda^3 \sigma^2$, where σ^2 is the GP signal variance (Hartikainen and Särkkä, 2010). We shall refer to the state vector of the GP $g(t)$ and its derivative in time $\dot{g}(t)$ as $z(t)$. The stationary covariance of the state is

$$P_\infty = \begin{bmatrix} \sigma^2 & 0 \\ 0 & \lambda^2 \sigma^2 \end{bmatrix}. \quad (6)$$

This SDE can be discretized to form a discrete-time state-space model over states $z_t = [g_t, \dot{g}_t]^\top$. Given the inter-point distance Δ_i between consecutive points in the chain, the discrete-time transition and process noise matrices are

$$A_i = \exp \left(\begin{bmatrix} 0 & 1 \\ -\lambda^2 & -2\lambda \end{bmatrix} \Delta_i \right), \quad Q_i = P_\infty - A_i P_\infty A_i^\top. \quad (7)$$

The observation model extracts the function value from the state through the observation vector $h = [1, 0]^\top$, so that $g_t = h^\top z_t$. This yields a linear-Gaussian state-space model equivalent to the GP, with $\mathcal{O}(N)$ inference via Bayesian filtering and smoothing (Särkkä, 2013).

3.3 Factor Graphs and Message Passing

Factor graphs provide a unified language for probabilistic inference, subsuming belief propagation, variational

inference and expectation propagation as instances of message computation on a graph (Kschischang et al., 2001; Şenöz et al., 2021; Heess et al., 2013). A large number of classical algorithms can be expressed as message passing routines on specific factor graphs, ranging from Kalman filtering for state estimation to linear-quadratic Gaussian control and dynamic programming for path planning (Loeliger et al., 2007; Palmieri et al., 2022). Message passing is by nature distributed and modular. By leveraging the factorization structure of a probabilistic model, one obtains efficient and explainable implementations of inference algorithms.

In Forney-style factor graphs, factors are nodes and variables are edges (Loeliger, 2004). An edge can be connected to at most two nodes. Messages are passed from and towards nodes where they are updated using node-specific update rules. For example, consider the factorized probabilistic model

$$p(x_1, x_2, x_3) = \underbrace{p(x_1 | x_2)}_{f_A} \underbrace{p(x_2 | x_3)}_{f_B} \underbrace{p(x_3)}_{f_C}, \quad (8)$$

and assume that we are interested in the marginal distribution of x_3 , given by

$$p(x_3) = \iint p(x_1, x_2, x_3) dx_1 dx_2. \quad (9)$$

Due to the factorization of Eq. 8, we can optimize the amount of computations by using the distributive law (i.e., by moving factors over the integration signs), leading to:

$$p(x_3) = \underbrace{\int f_A(x_1, x_2) dx_1}_{\vec{v}(x_2)} \underbrace{\int f_B(x_2, x_3) dx_2}_{\vec{v}(x_3)} \underbrace{f_C(x_3)}_{\check{v}(x_3)}. \quad (10)$$

In this way, the three-dimensional integral of Eq. 9 reduces to (multiplications of the results of) much simpler one-dimensional integrals. In the context of an FFG, the results of these sub-integrals can be interpreted as locally computable messages as depicted in Fig. 1.

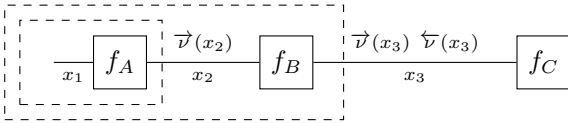


Figure 1: A FFG corresponding to the model given by Eq. 8.

This example illustrates the idea of the *sum-product rule* which states that for a generic node $f(y, x_1, \dots, x_n)$, the outgoing message $\vec{v}(y)$ is given by

$$\vec{v}(y) = \int \cdots \int f(y, x_1, \dots, x_n) \prod_{i=1}^n \vec{v}(x_i) dx_i, \quad (11)$$

where $\vec{v}(x_i)$ are incoming messages to the node. A terminal node, such as f_C in the example, has no incoming messages and will pass its factor as outgoing message. This is the basis of Bayesian inference in factor graphs; priors are terminal nodes, likelihoods are factors that pass observations backwards, and marginals are posterior distributions (Şenöz et al., 2021).

A key property of Forney-style factor graphs is compositionality: model changes, such as adding or removing observation factors, require only local modifications to the graph, not changes to the inference algorithm. This compositionality is central to our treatment of partial observations.

3.4 Declarative and reactive programming

Declarative programming is a paradigm that focuses on *what* a program should accomplish rather than *how* to achieve it, enabling developers to express logic at a higher level of abstraction (Antoy and Hanus, 2005). Reactive programming is a subset of declarative programming that adds time as a dimension (Bainomugisha et al., 2013). It declares what should happen when an event such as a data observation occurs, and avoids computation otherwise. This paradigm is particularly valuable in probabilistic contexts, where reactive probabilistic programming allows models to express uncertainty and perform inference in real time as new data streams in (Baudart et al., 2020; Bagaev and De Vries, 2023; Dos Martires et al., 2024). One of the implicit benefits is that a large amount of compilation overhead is avoided, which allows probabilistic models to scale to larger sizes. We shall utilize reactive probabilistic programming to exploit the specific structure of the state-space GP factor graph and obtain efficient message passing algorithms.

4 Message Passing-Based Multi-Output GP Inference

This section describes the proposed inference framework. We specify a multi-output state-space GP in factor graph form (Section 4.1) and explain how to infer states under partial observation (Section 4.2). In sequential-design applications, the state estimates are fed to a task-specific acquisition rule (Section 4.3) which selects the next query location x_+ .

4.1 Model specification

We first describe a nearest neighbor chain-based ordering for fixed low-dimensional candidate sets (Section 4.1.1), and then construct a factor graph based on a state-space GP with LMC (Section 4.1.2).

4.1.1 Nearest-Neighbor Chain Ordering

Extending state-space GPs beyond one-dimensional inputs is an active research area. In spatiotemporal settings, a natural temporal ordering exists, and the spatial structure can be handled separately via inducing points (Särkkä et al., 2013; Tebbutt et al., 2021; Hamelijnck et al., 2021). An alternative approach is the SPDE method of Lindgren et al. (2011), which discretizes the stochastic partial differential equation on a mesh to obtain a sparse precision matrix for Matérn fields in

arbitrary dimensions. Recently, Li et al. (2024) proposed a universal method for converting any stationary temporal GP into a state-space model via spectral factorization. Our work differs from these spatiotemporal approaches in that we do not assume separable structure; instead, we construct a one-dimensional chain via nearest-neighbor ordering. This makes state-space inference applicable to non-temporal candidate sets, but it is an approximation whose fidelity depends on how well the chain preserves the relevant geometry.

For M -dimensional inputs $X \in \mathbb{R}^{C \times M}$, with fixed candidate-set size C , we construct this ordering via a greedy nearest-neighbor heuristic. Let $\pi : \{1, \dots, C\} \rightarrow \{1, \dots, C\}$ denote the resulting permutation, so that $x_{\pi(i)}$ is the i -th point visited in the chain:

1. Start from an arbitrary point (e.g., x_1).
2. At each step, visit the closest unvisited point in Euclidean distance.
3. Record $\Delta_i = \|x_{\pi(i)} - x_{\pi(i-1)}\|$ as the inter-point distances.

The distances are normalized by their median, $\Delta_i \leftarrow \Delta_i / \text{median}(\Delta)$, and serve as the time steps in the state-space model. Points close in the original input space have small Δ_i (tight coupling), while distant points have large Δ_i (weak coupling, approaching the prior).

This chain ordering projects M -dimensional structure onto a one-dimensional sequence, introducing an approximation compared to the exact kernel matrix: only adjacent-point correlations are modeled directly, and long-range correlations are mediated through the Markov chain. We therefore expect fidelity to be strongest when the input dimension is low and local nearest-neighbor distances capture most of the relevant covariance structure. The brute-force greedy chain construction costs $\mathcal{O}(C^2)$; spatial indexing can reduce nearest-neighbor search costs in favorable low-dimensional settings.

4.1.2 State-Space GP with LMC

In state-space form, the full state $z_i \in \mathbb{R}^{2L}$ is the concatenation of all latent states. The system matrices are block-diagonal:

$$A_i = \text{blkdiag}(A_i^{(1)}, \dots, A_i^{(L)}), \quad (12)$$

$$Q_i = \text{blkdiag}(Q_i^{(1)}, \dots, Q_i^{(L)}), \quad (13)$$

$$P = \text{blkdiag}(P_\infty^{(1)}, \dots, P_\infty^{(L)}). \quad (14)$$

The observation matrix combines the mixing matrix W with the per-GP observation vectors:

$$H = [W_{:,1} h^{(1)\top} \quad \dots \quad W_{:,L} h^{(L)\top}] \in \mathbb{R}^{D \times 2L}, \quad (15)$$

so that $m_i = H z_i$ yields a D -dimensional deterministic observation mean.

Each output dimension is observed independently with per-output scalar noise:

$$y_{i,d} \sim \mathcal{N}(e_d^\top m_i, \tau_d^{-1}), \quad d = 1, \dots, D, \quad (16)$$

where e_d is the d -th standard basis vector in $\mathbb{R}^D$ and τ_d is the fixed noise precision for output d . This per-output scalar formulation keeps the model fully linear-Gaussian, enabling exact single-pass inference without variational approximations.

The full generative model is expressed as a factor graph for message passing (see Fig. 2):

$$z_0 \sim \mathcal{N}(0, P), \quad (17)$$

$$z_i \sim \mathcal{N}(A_i z_{i-1}, Q_i), \quad i = 1, \dots, C, \quad (18)$$

$$m_i = H z_i, \quad (19)$$

$$y_{i,d} \sim \mathcal{N}(e_d^\top m_i, \tau_d^{-1}), \quad d = 1, \dots, D. \quad (20)$$

Each output at each chain position is an independent scalar observation factor. Missing observations ($y_{i,d} = \text{missing}$) are handled natively: the corresponding factor contributes no message, and predictions flow through the graph unaffected.

4.2 Inference

Because all noise precisions τ_d are fixed scalars, the model is fully linear-Gaussian. Inference is a single forward-backward pass (exact Kalman smoother), automated by RxInfer.jl: no variational iterations, no constraints, no initialization. After the candidate chain has been constructed, the per-step inference cost is $\mathcal{O}(CDL)$: the block-diagonal structure of A_i and Q_i keeps the latent prior factorized across the L latent processes, and the D per-output scalar observation factors at each chain position contribute $\mathcal{O}(DL)$ message work via local updates through the deterministic mixing factor H , rather than dense operations on the joint $2L \times 2L$ state covariance. With partial observations, the effective D at each chain position may be smaller, reducing the cost further. This is the fundamental advantage over kernel-matrix methods, whose dense linear algebra cost grows polynomially, and in the naive fully coupled case cubically, with the number of observed output-point pairs.

Partial Observations via Modularity In multi-output settings, it is common for different outputs to be observed at different inputs. In sensor networks, for instance, sensors at different locations measure different quantities at different times (Liu et al., 2015). We call this *partial observation*: at each input i , only a subset of the D outputs may be available.

The factor graph formulation handles partial observations naturally. Each output $y_{i,d}$ corresponds to an independent scalar observation factor in the Forney-style factor graph. When $y_{i,d}$ is missing, the corresponding factor simply contributes no observation message. No covariance matrix restructuring is needed and no code path changes. The same model and the same inference call produce posteriors regardless of the observation pattern. The local message count can decrease when outputs are missing, although fixed overheads may dominate at moderate D . This is visualized in Fig. 3, where in the event that $y_{t,2}$ and $y_{t,3}$ are unobserved, messages 5–8 need not be sent. This reduces

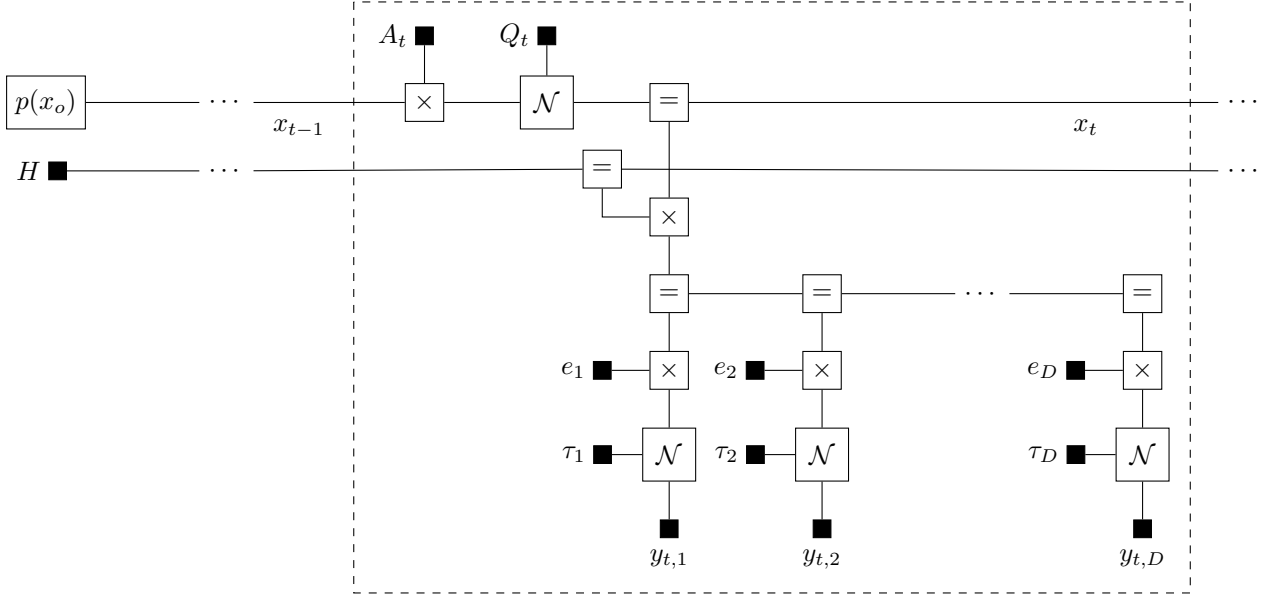


Figure 2: Forney-style Factor Graph of the model as described in Eqs. (17) to (20). The dashed box shows one timeslice of the subgraph that is repeated for each step t . The observation model shows D dimensions, which might or might not be observed.

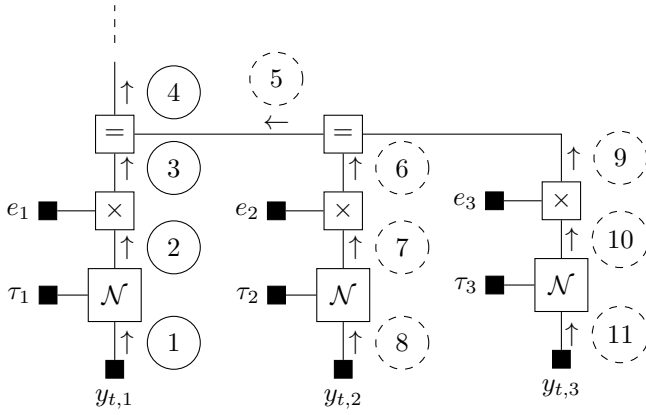


Figure 3: Example of message passing at a time point with partial observations: the dotted circles with 8 and 11 represent absent observations that lead to fewer message computations (all dotted circles).

preference vector $s \in \mathbb{R}^D$:

$$\mu_s(x) = s^\top \mu(x), \quad \varsigma_s(x) = \sqrt{\sum_{j=1}^D (s_j \sigma_j(x))^2}, \quad (21)$$

where $\mu(x), \sigma(x) \in \mathbb{R}^D$ are predicted mean and marginal standard-deviation vectors, with $\sigma_j(x) = \sqrt{\text{Var}[y_j | x]}$. The current acquisition uses predictive marginals, so $\varsigma_s(x)$ does not explicitly exploit cross-output covariance. The acquisition function is the Upper Confidence Bound:

$$\alpha^{\text{UCB}}(x) = \mu_s(x) + \beta \varsigma_s(x), \quad (22)$$

with exploration weight β . At each step, the unobserved candidate point with highest UCB is selected for evaluation.

5 Experiments

We compare the proposed state-space LMC (SS-LMC) with a kernel-matrix LMC (KM-LMC) on one-dimensional Bayesian quadrature and four-dimensional environmental monitoring BO. Both methods use identical LMC model structure: the same mixing matrix W , length-scales ℓ_l , signal variances σ_l^2 , and fixed diagonal noise. The comparison isolates the effect of the inference method and candidate-chain approximation under controlled hyperparameters. We report direct posterior-quality metrics in addition to sequential-design performance.

Implementation We use RxInfer.jl for reactive message passing (RMP)¹ (Bagaev et al., 2023). All timings compare CPU implementations in the same pipeline on a MacBook Pro with Apple M1 Pro CPU (8 cores) and 32 GB RAM.

¹Experiment code available at https://anonymous.4open.science/r/ProbNum_2026_BayesOptMP-4D04

5.1 Environmental-monitoring Bayesian optimization

BO comparison. We compare SS-LMC and KM-LMC under full observation, where all D outputs are observed at every point, and partial observation, where even-indexed points observe outputs $1:D/2$ and odd-indexed points observe outputs $D/2+1:D$.

Benchmark function. We use an environmental monitoring model based on pollutant diffusion (Bliznyuk et al., 2008; Maddox et al., 2021). It models diffusion from a point source in a one-dimensional medium with $M=4$ physical input parameters (source mass, diffusivity, source location, decay time) and $D=12$ outputs (3 spatial locations $\times$ 4 time points).

Configuration. We draw $C=500$ candidate points once at the start from a standard normal distribution and run 150 BO steps with $n_{\text{seed}}=10$ initial observations, averaged over 20 random seeds. BO always selects from the remaining unobserved candidates. The LMC uses $L=6$ latent GPs with length-scales $\ell_l \in \{1.0, 1.5, 2.0, 2.5, 3.0, 3.5\}$ and signal variances $\sigma_l^2 \in \{3.0, 3.0, 2.0, 2.0, 1.0, 1.0\}$. Per-output noise precision is $\tau_d = 1/0.2 = 5.0$ for all outputs. The UCB exploration weight is $\beta = 2.0$ with uniform scalarization $s = [1/D, \dots, 1/D]$. All hyperparameters are fixed throughout optimization.

5.1.1 Results

Optimization quality. Figure 4 shows the best scalarized value as a function of BO step over 20 seeds, including a random-search baseline. KM-LMC (full) obtains the highest mean final best value (1.242), while SS-LMC (full) reaches 1.203, close to random search (1.184); the per-seed paired difference between SS-LMC and random search (mean 0.019, std 0.081 across 20 seeds) is within noise, and SS-LMC strictly exceeds random search in 11/20 seeds. Thus, on this $M=4$ task, SS-LMC’s surrogate-guided search neither yields a clear improvement over random selection nor matches the kernel-matrix baseline.

Posterior quality. Figures 6 and 7 report held-out posterior quality on unobserved candidates. SS-LMC (full) trails KM-LMC (full): final RMSE is 0.302 vs. 0.252, and final MNLL is -0.422 vs. -0.587 (lower is better), showing posterior-fidelity loss at $M=4$.

Computational cost. Figure 5 shows median per-step wall-clock time over 150 BO steps. SS-LMC stays nearly constant, while KM-LMC grows with the kernel matrix. SS-LMC (full) takes ~ 112 s vs. ~ 1340 s for KM-LMC (full), an order-of-magnitude reduction (Table 1).

Partial observations. KM-LMC (partial) is substantially faster than KM-LMC (full) (294 s vs. 1340 s), since fewer observed entries reduce the kernel matrix size.

Table 1: Results on the environmental monitoring BO benchmark ($M=4$, $D=12$, $C=500$, 150 steps, 20 seeds). Best scalarized value and total wall-clock time are averaged over seeds; RMSE and MNLL are final held-out posterior-quality medians.

Method	Best	RMSE	MNLL	Time
SS-LMC (full)	1.203	0.302	-0.422	112 s
SS-LMC (partial)	1.206	0.339	-0.307	110 s
KM-LMC (full)	1.242	0.252	-0.587	1340 s
KM-LMC (partial)	1.231	0.312	-0.424	294 s
Random	1.184	—	—	—

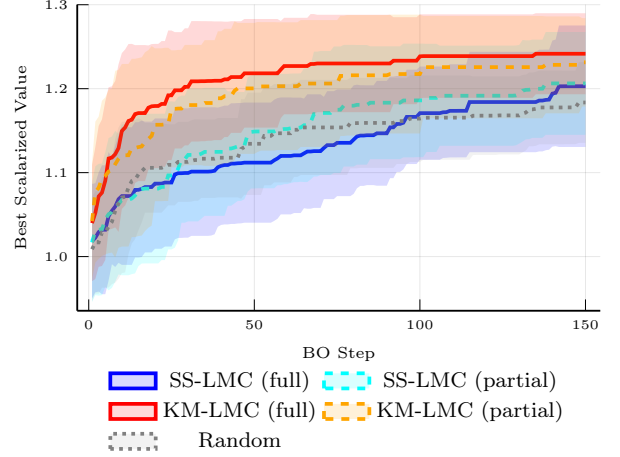


Figure 4: Convergence of the best scalarized value over 150 BO steps (mean $\pm$ std over 20 seeds). SS-LMC improves over random search but remains below KM-LMC (full) on this $M=4$ benchmark.

However, this speedup requires a fundamentally different code path that constructs variable-size covariance matrices from observed output–point pairs. SS-LMC (full) and SS-LMC (partial) have similar timing (~ 111 s), because reduced local messages are small relative to fixed overheads at $D = 12$. The main benefit is handling partial observations without covariance-matrix restructuring.

5.2 Bayesian quadrature

Configuration For BQ, we use a random multi-sine function with $M = 1$, $D = 6$, and integration range $(0, 2\pi]$. Here the chain is the natural one-dimensional ordering. We use $C = 200$, 150 steps, 20 seeds, and uncertainty sampling, selecting the unobserved candidate with maximum $\sum_{d=1}^D \sigma_d^2(x)$. The LMC uses $L = 3$, $\ell_l \in \{1.0, 1.5, 2.0\}$, $\sigma_l^2 \in \{3.0, 2.0, 1.0\}$, and $\tau_d = 10$.

Results Figures 9 and 10 show quadrature and posterior quality metrics. In this implementation, SS-LMC obtains lower final integral error (0.194 vs. 0.279) and RMSE (0.308 vs. 0.443) than KM-LMC, and reaches 150 steps faster (20.8 s vs. 63.4 s; Fig. 8).

Table 2: Per-step computational cost of multi-output GP inference. C : candidate chain length (fixed), N : number of observations (growing), D : output dimensions, L : latent GPs, P : observed output–point pairs.

	Naive MOGP	KM-LMC	SS-LMC (ours)
Per-step inference	$\mathcal{O}((DN)^3)$	$\mathcal{O}(N^3 + DL^2)$	$\mathcal{O}(CDL)$
Chain construction	—	—	$\mathcal{O}(C^2)$ brute force
Partial observations	$\mathcal{O}((DN)^3)$	$\mathcal{O}(P^3)$	$\mathcal{O}(CDL)$ upper bound
Grows with N ?	Yes	Yes	No, for fixed C
Total over N steps	$\mathcal{O}(D^3N^4)$	$\mathcal{O}(N^4 + NDL^2)$	$\mathcal{O}(C^2 + NCDL)$

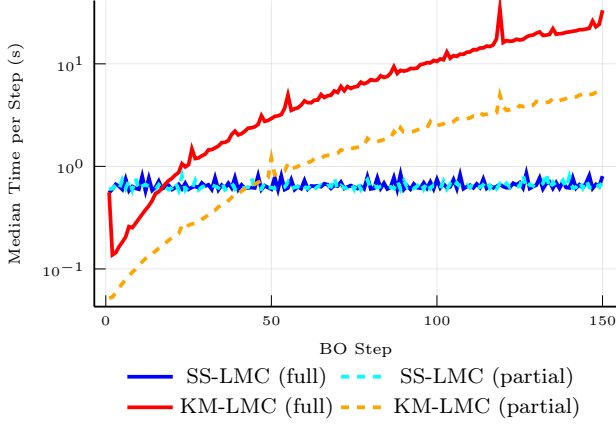


Figure 5: Median per-step wall-clock time (log scale) over 150 BO steps and 20 seeds. SS-LMC maintains near-constant cost while KM-LMC cost grows with the number of observed output–point pairs.

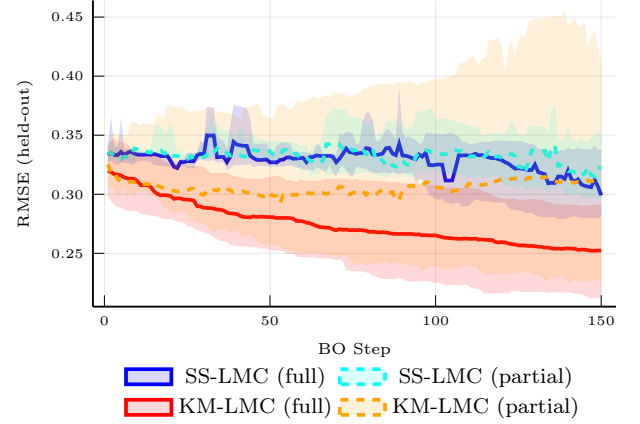


Figure 6: Held-out RMSE on unobserved candidates during the environmental-monitoring BO experiment (median and IQR over 20 seeds).

6 Discussion

Scaling advantage. After chain construction, SS-LMC has $\mathcal{O}(CDL)$ per-step inference for fixed C (Table 2), while kernel-matrix costs grow with observed output–point pairs. On the $D=12$ benchmark, this gives an order-of-magnitude lower total computation (Fig. 5 and Table 1); BQ shows the same effect through more steps per time budget (Fig. 8).

Approximation quality. The nearest-neighbor chain projects M -dimensional correlations onto a one-dimensional Markov chain, modeling adjacent correlations directly and mediating long-range correlations through the chain. The metrics show the tradeoff: at $M=1$, SS-LMC obtains lower final BQ error and RMSE than KM-LMC in our implementation (Figs. 9 and 10); at $M=4$, the chain compresses the input geometry into a one-dimensional Markov sequence, and SS-LMC’s final best value (1.203) is close to random search (1.184) and below KM-LMC (1.242), with worse RMSE and MNLL than KM-LMC (Figs. 4, 6 and 7), consistent with posterior fidelity deteriorating as input dimension increases. We therefore present the method as strongest for low-input-dimensional candidate sets, not as posterior-exact evidence from BO trajectories alone.

Compositionality. The partial observation experiment illustrates a broader point about factor graph compositionality. The same compositionality that lets miss-

ing observations omit local observation factors without covariance-matrix restructuring also applies to other model extensions. Online noise learning via conjugate priors, input-dependent noise, or non-stationary kernels could be incorporated by modifying local factors, without changing the inference algorithm. We leave such extensions to future work.

Limitations. The current framework uses fixed hyperparameters (W, ℓ_l, σ_l^2) throughout optimization. Hyperparameter tuning via marginal likelihood optimization within the state-space framework is possible (Durand et al., 2019) but not yet integrated. The nearest-neighbor ordering is greedy and path-dependent. A 5-start $\times$ 20-seed ablation suggests modest final-step variability (best-value std. 0.011, RMSE std. 0.013, MNLL std. 0.054), but path dependence remains. An input-dimension sweep is ongoing. We expect posterior fidelity to deteriorate as the chain compresses increasingly high-dimensional geometry into a one-dimensional Markov sequence. The method is currently limited to Matérn-class covariance functions that admit state-space representations.

7 Conclusion

We presented low-input-dimensional multi-output GP inference as message passing on a Forney-style factor graph, combining state-space GPs, LMC, and reactive message passing. On one-dimensional BQ, the method

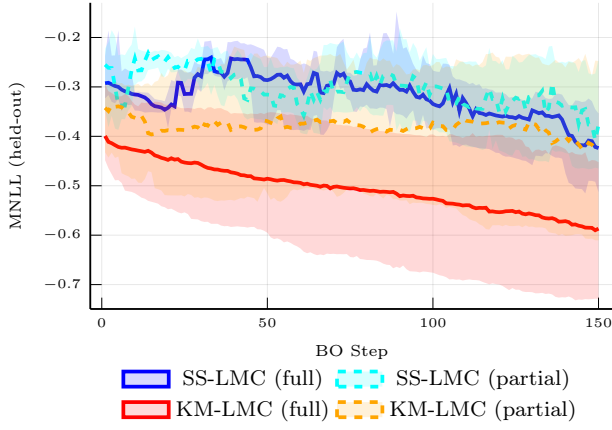


Figure 7: Held-out MNLL on unobserved candidates during the environmental-monitoring BO experiment (median and IQR over 20 seeds; lower is better).

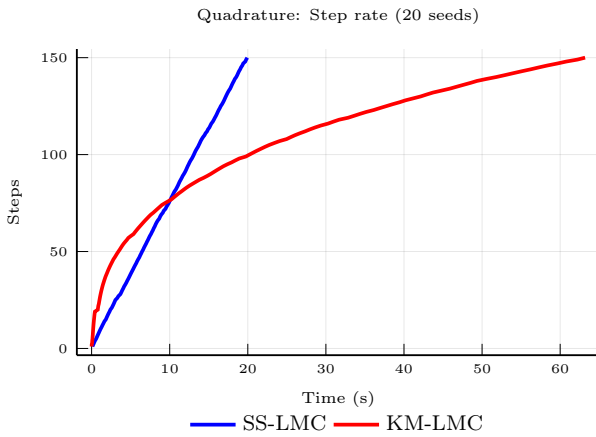


Figure 8: Median number of completed quadrature steps as a function of wall-clock time over 150 steps and 20 seeds. SS-LMC maintains a higher step rate than KM-LMC after the initial overhead.

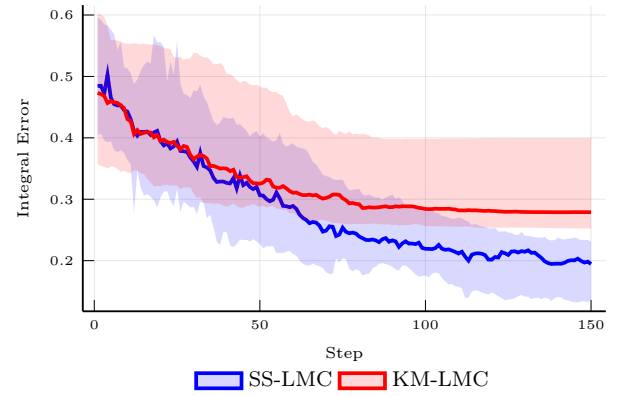


Figure 9: Bayesian quadrature integral error over 150 steps (median and IQR over 20 seeds; lower is better).

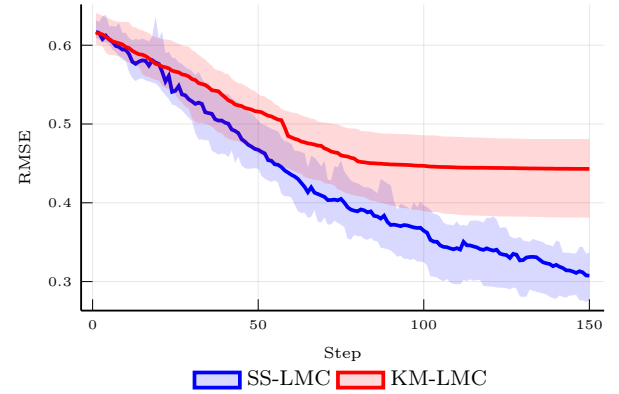


Figure 10: Bayesian quadrature RMSE over 150 steps (median and IQR over 20 seeds; lower is better).

obtains lower final integral error and RMSE than KM-LMC in our implementation while reducing wall-clock time. On four-dimensional environmental-monitoring BO, it gives an order-of-magnitude lower total computation and beats random search, but trails KM-LMC in held-out posterior fidelity. This supports the revised scope: a scalable approximation for low-input-dimensional MOGP inference, with BO and BQ as application studies. The factor graph handles partial observations without covariance-matrix restructuring.

References

- M. A. Alvarez and N. D. Lawrence. Computationally efficient convolved multiple output Gaussian processes. *Journal of Machine Learning Research*, 12: 1459–1500, 2011.
- M. A. Alvarez, L. Rosasco, and N. D. Lawrence. Kernels for vector-valued functions: A review. *Foundations and Trends® in Machine Learning*, 4(3):195–266, 2012.
- S. Antoy and M. Hanus. Declarative programming with

- function patterns. In *International Symposium on Logic-Based Program Synthesis and Transformation*, pages 6–22, 2005.
- D. Bagaev and B. De Vries. Reactive Message Passing for Scalable Bayesian Inference. *Scientific Programming*, 2023:1–26, 2023. doi: 10.1155/2023/6601690.
- D. Bagaev, A. Podusenko, and B. de Vries. RxInfer: A Julia package for reactive real-time Bayesian inference. *Journal of Open Source Software*, 8(84):5161, Apr. 2023. ISSN 2475-9066. doi: 10.21105/joss.05161.
- E. Bainomugisha, A. L. Carreton, T. v. Cutsem, S. Mostinckx, and W. d. Meuter. A survey on reactive programming. *ACM Computing Surveys*, 45(4):1–34, 2013.
- G. Baudart, L. Mandel, E. Atkinson, B. Sherman, M. Pouzet, and M. Carbin. Reactive probabilistic programming. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 898–912, 2020.
- N. Bliznyuk, D. Ruppert, C. Shoemaker, R. Regis, S. Wild, and P. Mugunthan. Bayesian calibration and uncertainty analysis for computationally expensive models using optimization and radial basis function approximation. *Journal of Computational and Graphical Statistics*, 17(2):270–294, 2008. doi: 10.1198/106186008X320681.

- 557 E. V. Bonilla, K. Chai, and C. Williams. Multi-task
558 Gaussian Process Prediction. In *Advances in Neural*
559 *Information Processing Systems*, volume 20, 2007.
- 560 W. Bruinsma, E. Perim, W. Tebbutt, S. Hosking,
561 A. Solin, and R. Turner. Scalable Exact Inference in
562 Multi-Output Gaussian Processes. In *International*
563 *Conference on Machine Learning*, pages 1190–1201,
564 2020.
- 565 P. Z. Dos Martires, L. De Raedt, and A. Kim-
566 mig. Declarative probabilistic logic programming in
567 discrete-continuous domains. *Artificial Intelligence*,
568 337:104227, 2024.
- 569 N. Durrande, V. Adam, L. Bordeaux, S. Eleftheriadis,
570 and J. Hensman. Banded Matrix Operators for Gaus-
571 sian Markov Models in the Automatic Differentiation
572 Era. In *International Conference on Artificial Intel-*
573 *ligence and Statistics*, pages 2780–2789, 2019.
- 574 P. I. Frazier. Bayesian optimization. In *Recent advances*
575 *in optimization and modeling of contemporary prob-*
576 *lems*, pages 255–278. Inform, 2018.
- 577 R. Garnett. *Bayesian Optimization*. Cambridge Uni-
578 versity Press, 2023.
- 579 O. Hamelijnck, W. Wilkinson, N. Loppi, A. Solin, and
580 T. Damoulas. Spatio-temporal variational gaussian
581 processes. *Advances in Neural Information Process-*
582 *ing Systems*, 34:23621–23633, 2021.
- 583 J. Hartikainen and S. Särkkä. Kalman filtering and
584 smoothing solutions to temporal Gaussian process re-
585 gression models. In *IEEE International Workshop on*
586 *Machine Learning for Signal Processing*, pages 379–
587 384, 2010.
- 588 N. Heess, D. Tarlow, and J. Winn. Learning to pass
589 expectation propagation messages. In *Advances in*
590 *Neural Information Processing Systems*, volume 26,
591 2013.
- 592 P. Hennig and C. J. Schuler. Entropy search for
593 information-efficient global optimization. *Journal of*
594 *Machine Learning Research*, 13(6), 2012.
- 595 F. R. Kschischang, B. J. Frey, and H.-A. Loeliger. Fac-
596 tor graphs and the sum-product algorithm. *IEEE*
597 *Transactions on Information Theory*, 47(2):498–519,
598 2001.
- 599 W. Li, Y. Wang, C. Li, and A. Wu. Markovian
600 Gaussian process: A universal state-space repre-
601 sentation for stationary temporal Gaussian process.
602 *arXiv.2407.00397*, 2024.
- 603 F. Lindgren, H. Rue, and J. Lindström. An explicit link
604 between Gaussian fields and Gaussian Markov ran-
605 dom fields: The stochastic partial differential equa-
606 tion approach. *Journal of the Royal Statistical So-*
607 *ciety: Series B (Statistical Methodology)*, 73(4):423–
608 498, 2011. ISSN 1467-9868. doi: 10.1111/j.1467-9868.
609 2011.00777.x.
- Q. Liu, Z. Wang, X. He, and D.-H. Zhou. Event-based
recursive distributed filtering over wireless sensor net-
works. *IEEE Transactions on Automatic Control*, 60
(9):2470–2475, 2015.
- H.-A. Loeliger. An introduction to factor graphs. *IEEE*
Signal Processing Magazine, 21(1):28–41, 2004. doi:
10.1109/MSP.2004.1267047.
- H.-A. Loeliger, J. Dauwels, J. Hu, S. Korl, L. Ping, and
F. R. Kschischang. The Factor Graph Approach to
Model-Based Signal Processing. *Proceedings of the*
IEEE, 95(6):1295–1322, 2007. ISSN 0018-9219. doi:
10.1109/JPROC.2007.896497.
- W. J. Maddox, M. Balandat, A. G. Wilson, and E. Bak-
shy. Bayesian optimization with high-dimensional
outputs. In *Advances in Neural Information Process-*
ing Systems, volume 34, pages 19274–19287, 2021.
- F. A. N. Palmieri, K. R. Pattipati, G. D. Gennaro,
G. Fioretti, F. Verolla, and A. Buonanno. A unifying
view of estimation and control using belief propaga-
tion with application to path planning. *IEEE Access*,
10:15193–15216, 2022.
- C. E. Rasmussen and C. K. I. Williams. *Gaussian Pro-*
cesses for Machine Learning. MIT Press, 2006.
- S. Särkkä. *Bayesian Filtering and Smoothing*. Cam-
bridge University Press, 2013.
- S. Särkkä and A. Solin. *Applied Stochastic Differential*
Equations. Cambridge University Press, 2019.
- S. Särkkä, A. Solin, and J. Hartikainen. Spatiotem-
poral learning via infinite-dimensional Bayesian fil-
tering and smoothing: A look at Gaussian process
regression through Kalman filtering. *IEEE Signal*
Processing Magazine, 30(4):51–61, July 2013. doi:
10.1109/MSP.2013.2246292.
- İ. Şenöz, T. van de Laar, D. Bagaev, and B. de Vries.
Variational Message Passing and Local Constraint
Manipulation in Factor Graphs. *Entropy*, 23(7), June
2021. ISSN 1099-4300. doi: 10.3390/e23070807.
- A. J. Smola, S. V. N. Vishwanathan, and E. Eskin.
Laplace propagation. In *Advances in Neural Infor-*
mation Processing Systems, pages 441–448, 2004.
- J. Snoek, H. Larochelle, and R. P. Adams. Practi-
cal Bayesian optimization of machine learning algo-
rithms. *Advances in Neural Information Processing*
Systems, 25, 2012.
- N. Srinivas, A. Krause, S. Kakade, and M. Seeger. Gaus-
sian process optimization in the bandit setting: No
regret and experimental design. In *International Con-*
ference on Machine Learning, pages 1015–1022, 2010.
- W. Tebbutt, A. Solin, and R. E. Turner. Combin-
ing pseudo-point and state space approximations for
sum-separable Gaussian processes. In *Conference*
on Uncertainty in Artificial Intelligence, pages 1607–
1617. PMLR, 2021.